

2.11 Math module ^

Settings

Beta features

math module

Python comes with a standard math module to support such advanced

module

A module is Python code located in another file.

Figure 2.11.1: Importing the math module and calling a math module function

```
import math

num = 49
num_sqrt = math.sqrt(num)
```

function

A function is a list of statements that can be executed simply by referencing its name.

function call

The process of invoking a function is referred to as a function call.

argument

The item passed to a function is referred to as an argument.

Try 2.11.1: Example of using a math function: Savings interest program.

Note: Blank print statements are used to go to the next line after reading pre-entered input.



▶ Run

```
1  import math
2
3  base = float(input("Enter initial savings: "))
4  print()
5
6  rate = float(input("Enter annual interest % rate: "))
7  print()
8
9  years = int(input("Enter years that pass: "))
10 print()
11
12 total = base * math.pow(1 + (rate / 100), years)
13
14 print(f"Savings after {years} years is ${total:.2f}")
```

CONSOLE



Table 2.11.1: Functions in the standard math module.

Function	Description	Function	
Number representation and theoretic functions			
ceil(x)	Round-up value	fabs(x)	Absolute value
factorial(x)	factorial ($3! = 3 * 2 * 1$)	floor(x)	Round-down
fmod(x, y)	Remainder of division	fsum(x)	Floating-point sum
Power, exponential, and logarithmic functions			
exp(x)	Exponential function e^x	log(x, (base))	Natural logarithm
pow(x, y)	Raise x to power y	sqrt(x)	Square root
Trigonometric functions			
acos(x)	Arc cosine	asin(x)	Arc sine
atan(x)	Arc tangent	atan2(y, x)	Arc tangent
cos(x)	Cosine	sin(x)	Sine
hypot(x1, x2, x3, ..., xn)	Length of vector from origin	degrees(x)	Convert from radians to degrees
radians(x)	Convert degrees to radians	tan(x)	Tangent
cosh(x)	Hyperbolic cosine	sinh(x)	Hyperbolic sine
Complex number functions			
gamma(x)	Gamma function	erf(x)	Error function
Mathematical constants			
pi (constant)	Mathematical constant 3.141592...	e (constant)	Mathematical constant 2.71828...

**CHALLENGE
ACTIVITY**

2.11.1: Math functions.

763650.1443318.qx3zqy7

Start

Type the program's output

```
import math  
  
x = math.sqrt(4.0)  
print(x)
```

2.0

1

2

3

4

5

Check**Next**View solution ▼ *(Instructors only)***CHALLENGE
ACTIVITY**

2.11.2: Using math functions to calculate the distance between two points.

763650.1443318.qx3zqy7

Organize the correct code statements to find the distance between point (x1, y1) and point (x2, y2), points_distance. The calculation is:

$$\text{Distance} = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$

[Click here for example](#) ▼

Note: Not all code statements on the left should be used in the final solution.

How to use this tool ▼

Unused

```
x_dist = math.pow((x1 - y1), 2)
x_dist = math.pow((x2 - x1), 2)
y_dist = math.pow((y2 - y1), 2)
y_dist = math.pow((x2 - y2), 2)
points_distance = math.sqrt(x_dist + y_dist)
```

main.py

```
import math

x1 = float(input())
y1 = float(input())
x2 = float(input())
y2 = float(input())

print(points_distance)
```

Check

View solution ▼ (Instructors only)

CHALLENGE ACTIVITY

2.11.3: Writing math calculations.

763650.1443318.qx3zqy7

Start

Compute: $\text{answer} = |b| - c$

[Click here for example](#) ▼

Note: Functions in the math module can be accessed using dot notation. Ex: The constant pi is

```
1 import math
2
3 b = float(input())
4 c = float(input())
5
6 """ Your code goes here """
7
8 print(f"answer = {answer:.1f}") # Outputs answer with 1 decimal place
```

1

2

3

Check**Next level**

View solution ▼ (Instructors only)